

Contact Book Management System — Project Report

1. Title Page

Project Title: Contact Book Management System

Language Used: C Language

Prepared By: Priyansh Jain

SAP ID: 590026142

Course : Programming in C

Submitted To: Dr. Srinivasan Ramachandran Sir

Date: 25th November ,2025

2. Abstract

This project presents a simple and efficient Contact Book Management System developed in the C programming language. The system allows users to add, view, search, and delete contact records stored in a text file named as Contact_list.txt . The project show the use of structures, file handling, conditional constructs, loops, and menu-driven programming in C. The report includes system design, algorithms, implementation details with snippets, testing, and conclusions with future improvements.

3. Problem Definition

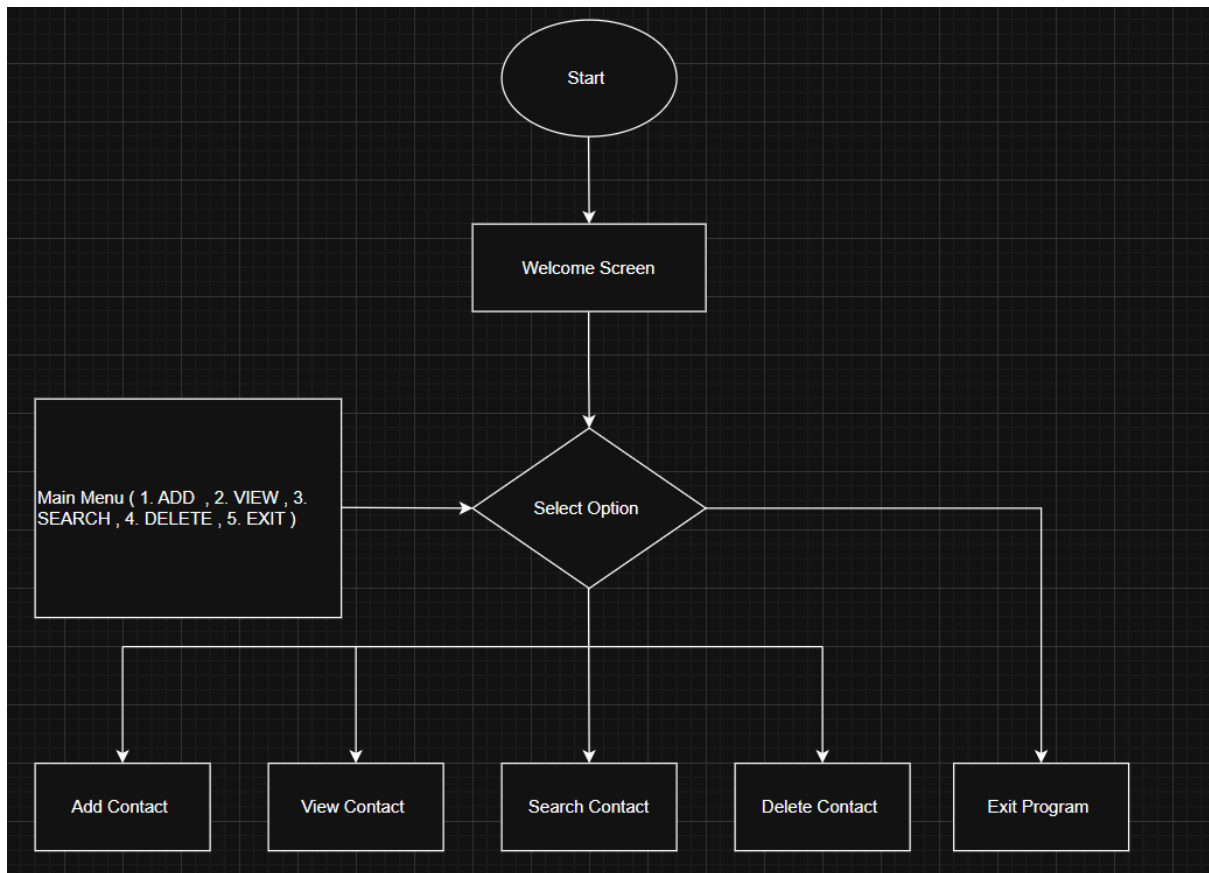
Managing contacts manually becomes difficult as the number increases. A digital solution is needed to:

- Store and manage contacts (Name, Phone Number)
 - Search a contact by name
 - Delete an existing contact
 - View all saved contacts
- Maintain data even after program termination using file handling

The aim of this project is to develop a user-friendly console-based contact management program in C.

4. System Design

4.1 Flowchart



4.2 Algorithms

Algorithm: Add Contact

1. First it will ask the user for contact name.
2. Then it will ask for phone number.
3. It should Validate if phone number is 10 digits.
4. Open file `contact_list.txt` in append mode.
5. Then it will Save the contact.
6. Close file and display contact save messages message.

Algorithm: View Contacts

1. Open file in read mode.

2. Read each line until EOF.
3. Display name and phone number.
4. Close the file.

Algorithm: Search Contact

1. Open file in read mode.
2. Input name to search.
3. Compare each saved name with search input.
4. If found, display details.
5. If not found, show "Contact Not Found".

Algorithm: Delete Contact

1. Open original file in read mode.
2. Open temporary file in write mode.
3. Copy all contacts except the one to delete.
4. Replace original file with temporary file.
5. Display success or error message.

5. Implementation Details (with Code Snippets)

Below are key parts of the provided C program:

5.1 Structure Definition

```
typedef struct Contact Book
{
    char name[50];
    char phone[20];
} CB;
```

5.2 Welcome Page Function

```
int FirstPage()
{
    printf("\n\n");
    printf("=====\\n");
    printf("\\t\\tWELCOME TO CONTACT BOOK\\t\\t\\t\\t\\n");
    printf("=====\\n");
    printf("                \\t\\t Developed in C Language\\n");
    printf("                \\t\\t  Save | Search | Delete\\n");
    printf("-----\\n");
    printf("                Loading.....");

    for (int i = 0; i < 6; i++)
    {
        printf(".");
        fflush(stdout);
        for (int j = 0; j < 200000000; j++)
            ;
    }
}
```

5.3 Add Contact

```
void addContact()
{
    struct CB;
    FILE *fp = fopen("contact/contact_list.txt", "a");

    if (fp == NULL)
    {
        printf("Error opening file!\n");
        return;
    }

    printf("Enter Name: ");
    scanf("%s", c.name);
    while (1)
    {
        printf("Enter Phone Number: ");
        scanf("%s", c.phone);
        printf("\nInvalid Phone Number! Please enter a 10-digit number.\n");
        if (strlen(c.phone) != 10)
        {
        }
        else
        {
            break;
        }
    }

    fprintf(fp, "%s %s\n", c.name, c.phone);
    fclose(fp);

    printf("Contact Added Successfully!\n");
}
```

5.4 View Contacts

```
void viewContacts()
{
    struct CB;
    FILE *fp = fopen("contact/contacts_list.txt", "r");

    if (fp == NULL)
    {
        printf("No contacts found!\n");
        return;
    }

    printf("\t\t\t\t\n--- Contact List ---\n");
    while (fscanf(fp, "%s %s", c.name, c.phone) != EOF)
    {
        printf("Name: %-15s Phone Number: %-15s\n", c.name, c.phone);
    }
    fclose(fp);
}
```

5.5 Search Contact

```
void searchContact()
{
    struct CB;
    char searchName[50];
    int found = 0;

    FILE *fp = fopen("contact/contacts_list.txt", "r");

    if (fp == NULL)
    {
        printf("No contacts found!\n");
        return;
    }

    printf("Enter name to search: ");
    scanf("%s", searchName);

    while (fscanf(fp, "%s %s", c.name, c.phone) != EOF)
    {
        if (strcmp(c.name, searchName) == 0)
        {
            printf("\nContact Found!\n");
            printf("Name: %s\nPhone Number: %s\n", c.name, c.phone);
            found = 1;
            break;
        }
    }

    if (!found)
        printf("Contact Not Found!\n");

    fclose(fp);
}
```

5.6 Delete Contact


```
void deleteContact()
{
    struct CB;
    char deleteName[50];
    int found = 0;

    FILE *fp = fopen("contact/contacts_list.txt", "r");
    FILE *temp = fopen("temp.txt", "w");

    if (fp == NULL)
    {
        printf("No contacts found!\n");
        return;
    }

    printf("Enter name to delete: ");
    scanf("%s", deleteName);

    while (fscanf(fp, "%s %s", c.name, c.phone) != EOF)
    {
        if (strcmp(c.name, deleteName) != 0)
        {
            fprintf(temp, "%s %s\n", c.name, c.phone);
        }
        else
        {
            found = 1;
        }
    }

    fclose(fp);
    fclose(temp);

    remove("contact/contacts_list.txt");
    rename("contact/temp.txt", "contact/contacts_list.txt");

    if (found)
        printf("Contact Deleted Successfully!\n");
    else
        printf("Contact Not Found!\n");
}
```

6. Testing & Results

Test Case	Input	Expected Output	Result
Add contact (valid contact)	Name: "Priyansh" Phone: "9876543210"	Contact added successfully	Passed
Add contact (invalid contact)	Phone: "1234"	Invalid phone number	Passed
View contacts	—	Contact list	Passed
Search contact	Priyansh	Contact found	Passed
Search contact (not exist)	Ayush	Contact Not Found	Passed
Delete contact	Piyush	Contact Deleted	Passed
Delete contact (not exist)	"Rohan"	Contact Not Found	Passed

7. Conclusion & Future Work

Conclusion

The Contact Book System successfully demonstrates file handling, structures, loops, and conditional statements in C. It provides essential contact management features using simple data storage techniques.

Future Enhancements

- Add email & address fields
- Allow updating existing contacts
- Use binary file storage for data security
 - Sort contacts alphabetically
 - Add GUI-based interface
- Allow full names with spaces

8. References

1. "Let us C " – **Yasha Vant Kanetkar**
2. Classroom Lecture Notes